

Backpropagation in Artificial Neural Networks

Conceptual notes • gradient computation • chain rule • memoization • training workflow

1. What is Backpropagation?

Backpropagation is an algorithm for efficiently computing the gradients of a neural network's loss function with respect to its weights and biases. It uses the **chain rule of calculus** and propagates gradient information from the loss backward through the computational graph.

Important: Backpropagation computes gradients; it does not itself perform the parameter update.

2. Complete Training Workflow

Forward pass → Loss calculation → Backpropagation → Gradients → Optimizer → Parameter update

Stage	What happens
1. Forward pass	Input moves through layers and produces a prediction ■.
2. Loss	The prediction is compared with the true target y .
3. Backpropagation	Gradients of the loss with respect to parameters are computed.
4. Optimizer	Gradient Descent, SGD, Adam, etc. use the gradients.
5. Update	Weights and biases are changed to reduce the loss.

3. Why Do We Need Gradients?

For a parameter w , $\partial L / \partial w$ tells us how sensitive the loss L is to a small change in w . Its sign indicates the direction in which the loss tends to change, while its magnitude indicates the strength of that sensitivity.

Gradient-descent update:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$$

Remember: The gradient is not the actual parameter update. The learning rate and optimizer determine how the parameter is changed.

4. Why Does Backpropagation Move Backward?

A neural network can be viewed as a computational graph. Earlier parameters influence later activations, which eventually influence the loss. Therefore, to find the effect of an earlier parameter on the loss, we use derivatives from later computations.

Example: $w \rightarrow z \rightarrow a \rightarrow \blacksquare \rightarrow L$

To calculate $\partial L / \partial w$, we need the chain of dependencies:

$$\partial L / \partial w = (\partial L / \partial a) \cdot (\partial a / \partial z) \cdot (\partial z / \partial w)$$

Thus, gradient information naturally travels from the loss toward earlier layers: **output** → **hidden layers** → **input**.

5. Chain Rule: The Mathematical Engine

The chain rule allows us to calculate the effect of a variable on the loss through multiple intermediate variables.

$$w \rightarrow z^1 \rightarrow z^2 \rightarrow z^3 \rightarrow L$$

$$\partial L / \partial w = (\partial L / \partial z^3) (\partial z^3 / \partial z^2) (\partial z^2 / \partial z^1) (\partial z^1 / \partial w)$$

Core idea: Backpropagation is an efficient way of applying the chain rule throughout the neural network.

6. Weight and Bias Gradients

For a neuron in layer l :

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}$$
$$\partial L / \partial w_{ij}^{(l)} = (\partial L / \partial z_j^{(l)}) \cdot a_i^{(l-1)}$$

A useful intuition is: **weight gradient = error signal $\times$ input to that weight.**

$$\partial L / \partial b_j^{(l)} = \partial L / \partial z_j^{(l)}$$

7. Memoization

Memoization is a general computational optimization technique in which previously computed results are stored so they can be reused instead of being recomputed. In gradient computation, intermediate values and derivatives may be needed by multiple calculations, so caching them avoids redundant work.

Compute once → Store → Reuse

Trade-off: memoization uses additional memory in exchange for reducing redundant computation and improving computational efficiency.

Memoization is not the mathematical definition of backpropagation. It is a general optimization idea that supports efficient computation by reusing intermediate results.

8. Backpropagation vs. Gradient Descent

Backpropagation	Gradient Descent / Optimizer
Computes gradients	Uses gradients
Uses the chain rule	Uses an optimization/update rule
"How does each parameter affect the loss?"	"How should I change the parameters?"
Gradient computation	Parameter optimization

9. Common Conceptual Mistakes to Avoid

- **Backpropagation is Gradient Descent.**
- Backpropagation computes gradients; the optimizer uses them to update parameters.
- **Backpropagation updates weights backward.**
- Backpropagation propagates gradient information backward. Parameter updates are a separate optimization step.
- **The gradient directly gives the exact update.**
- The gradient provides direction/sensitivity; the optimizer and learning rate determine the update.
- **Memoization is unique to backpropagation.**
- Memoization is a general technique for avoiding repeated computation.

10. Backpropagation — WHAT, WHY, and HOW

WHAT?

What is backpropagation? It is the process/algorithm used to compute the gradient of the loss with respect to the neural network's parameters (weights and biases) by propagating gradient information backward through the network.

What does it produce? For each trainable parameter, it produces quantities such as $\partial L / \partial w$ and $\partial L / \partial b$.

WHY?

Why do we need it? A neural network may contain thousands, millions, or billions of parameters. We need to know how each parameter contributes to the loss so that an optimizer can improve the model.

Why not calculate every derivative independently? Because that would repeatedly perform overlapping computations. Backpropagation exploits the chain rule and reuses intermediate results, making gradient

computation dramatically more efficient.

Why backward? The loss is at the end of the computational graph. Gradients needed for earlier parameters depend on later computations, so gradient information is propagated from the loss backward toward the input.

HOW?

Step	Process	Main idea
1	Forward pass	Compute activations and prediction ■.
2	Loss	Measure the difference between ■ and target y .
3	Start at loss	Compute the gradient of loss with respect to the output.
4	Chain rule	Move backward through each operation/layer.
5	Reuse results	Use stored intermediate values to avoid unnecessary recomputation.
6	Parameter gradients	Obtain $\partial L / \partial w$ and $\partial L / \partial b$ for all trainable parameters.
7	Optimizer	Use the gradients to determine parameter updates.
8	Repeat	Perform this process across batches/iterations until training progresses.

One mental model: WHAT = compute gradients. WHY = know how parameters affect loss so the model can improve. HOW = use the chain rule to propagate gradients backward, reuse intermediate results, and pass the resulting gradients to an optimizer.

11. The Complete Mental Model

1. Feed the input through the network.
2. Produce the prediction.
3. Compare prediction with the target and calculate loss.
4. Start from the loss and calculate gradients using the chain rule.
5. Propagate those gradients backward through the layers.
6. Obtain gradients for every relevant weight and bias.
7. Give those gradients to an optimizer.
8. Update the parameters and repeat for many training steps.

12. One-Sentence Definition

“Backpropagation uses the chain rule to efficiently propagate the loss gradient backward through a neural network, calculating how each weight and bias contributes to the loss; an optimizer then uses those gradients to update the parameters.”

13. What to Know Before Moving Deeper

- Understand forward propagation and how activations are produced.
- Understand loss functions and why lower loss is desirable.
- Understand derivatives and gradients intuitively.
- Understand the chain rule.
- Understand computational graphs and dependencies.
- Understand backpropagation as gradient computation.

- Clearly separate backpropagation from optimization.
- Understand weight and bias gradients.
- Understand memoization as a computational efficiency technique.
- Be able to trace gradient flow from output toward earlier layers.

Quick Recall

Backpropagation: Compute gradients.

Chain rule: Connect derivatives through dependencies.

Gradient: Tells how loss changes with a parameter.

Optimizer: Uses gradients to update parameters.

Memoization: Store results to avoid redundant computation.